Classical Optimizer Vs Quantum-Assisted Optimizer

성명	노미정
프로젝트명	Classical Optimizer와 Quantum-Assisted Optimizer의 메모리 및 깊은 모델 학습 비교
과제 유형	이론
한 줄 소개	동일한 Classical Neural Network(고전 신경망)에서 Classical Optimizer(고전 최적화 방식)와 Quantum-Assisted Optimizer(양자 보조 최적화 방식)를 비교하여 메모리 사용량과 깊은 모델의 학습 가능성을 평가한다.

① 주제와 배경	Deep Learning(딥러닝) 모델의 규모가 커지면 Weight(가중치), Activation(활성값), Gradient(기울기), Optimizer State(최적화기 상태)를 보관하기 위한 메모리 사용량이 증가한다. SGD(확률적 경사 하강법)와 Adam(적응적 모멘트 추정)은 Backpropagation(역전파)으로 기울기를 계산하며 중간 활성값과 기울기를 저장한다. Adam은 기울기의 First Moment(1차 모멘트)와 Second Moment(2차 모멘트) 상태도 추가로 유지한다. Gradient-Free Optimization(기울기 없는 최적화)은 역전파 없이 Loss Function(손실 함수)의 평가 결과로 가중치를 갱신할 수 있다. 역전파용 활성값과 기울기 저장을 줄일 가능성이 있지만 Direction Vector(방향 벡터), Temporary Weight(임시 가중치), Optimization Workspace(최적화 작업 공간)도 필요해 전체 메모리는 구현에 따라 달라진다. 동일한 Classical Neural Network(고전 신경망)에서 Classical Optimizer(고전 최적화 방식)와 Quantum Measurement Based Optimization(양자 측정 기반 최적화 방식)을 비교한다. Quantum Circuit Simulator(양자회로 시뮬레이터)의 측정값으로 탐색 방향을 생성하고 손실 평가에 활용하며, 같은 목표 품질을 고려해 메모리와 성능·시간·평가 횟수를 측정한다. 본 실험의 양자 보조 방식은 양자 측정으로 방향을 만들고 Classical Neural Network(고전 신경망)의 손실 평가와 파라미터 갱신을 고전 컴퓨터에서 수행하는 Hybrid Optimization(하이브리드 최적화)이다. 기울기 저장이 없더라도 방향 벡터와 임시 배열이 필요하므로 전체 메모리 절감은 측정으로 확인한다.
② 탐구 질문 또는 문제점	질문 ① Backpropagation(역전파)에서 계산한 Gradient(기울기)를 저장하지 않는 Quantum-Assisted Optimizer(양자 보조 최적화 방식)는 SGD·Adam에 비해 학습 과정의 시간과 최대 메모리 사용량을 줄이는가? 질문 ② Hidden Layer Depth(은닉층 깊이)가 증가할 때 Backpropagation(역전파)의 Gradient Vanishing(기울기 소실)과 학습 정체가 관찰되는가? 기울기를 역전파하지 않는 양자 보조 방식이 해당 조건에서 유효한 학습 대안이 되는가? 질문 ③ Quantum-Assisted Optimizer(양자 보조 최적화 방식) 대신 Classical-Direction COBYLA(난수 방향 COBYLA)도 메모리·학습 품질·실행 비용 면에서 대안이 되는가?
③ 실험 목표 또는 개선 목표	① 실험 설정 ■ Model = Classical Neural Network(고전 신경망) 입력 4개, 출력 2개의 Binary Classification(이진 분류)에 ReLU(렐루) 은닉층과 Cross-Entropy Loss(교차 엔트로피 손실)를 사용한다.

	출력은 Logit(로짓)이며 필요할 때 Softmax(소프트맥스)로 확률을 계산한다. ■ Hidden Layer Depth: 4×1, 8×2, 256×8·12·16·24·32·48·64 ■ Data : Sample(표본) 512 개를 80:20 으로 나누고 Data Seed(데이터 난수 시드) 2026 을 사용한다. ② 실험 방법 동일 모델·Seed(시드)에서 데이터와 초기 가중치를 고정한채, Optimizer 별로 비교한다. XPU(or GPU)를 사용불가한 device에서는 CPU 에서 실험 합니다. **비교 조건 및 학습 설정 보완** ● 목표 Training Loss(학습 손실) 0.65·0.55·0.20 에 각각 처음 도달할 때의 시간, 순전파 평가 횟수, 메모리를 기록한다. 미도달은 별도로 표시하고 Validation Loss(검증 손실)와 Validation Accuracy(검증 정확도)를 함께 확인한다. ● 같은 실행 시간, 같은 Forward Evaluation(순전파 평가) 횟수, 같은 목표 품질 도달 비용을 각각 비교한다. Backward Pass(역전파 연산)와 Circuit Sampling(회로 표본 수집) 비용도 함께 기록한다. ● SGD·Adam 의 학습 횟수와 Learning Rate(학습률)를 조정하여 손실 정체를 확인한다. Initialization(초기화), Normalization(정규화), Residual Connection(잔차 연결)을 적용한 모델은 기본 모델과 구분하여 비교한다. **양자 방향 비교 및 비용 측정** ● 동일한 탐색 구조와 평가 예산에서 Classical Random Direction(고전 난수 방향)과 Quantum-Sampled Direction(양자 표본 방향)을 비교한다. ● Circuit Execution Count(회로 실행 횟수), Shot Count(측정 횟수), 실행 시간과 시뮬레이터 메모리를 가능한 범위에서 기록한다. 실제 양자 장치의 대기·통신·잡음 영향은 별도 실험으로 검증한다. ③ 관찰값: Training/Validation Loss(학습·검증 손실), Validation Accuracy(검증 정확도), Forward Evaluation(순전파 평가) 횟수, Elapsed Time(경과 시간), CPU RSS(중앙처리장치 상주 메모리)와 XPU Peak Allocated/Reserved Memory(XPU 최대 할당·확보 메모리)를 기록한다. Backpropagation(역전파)의 Gradient Vanishing(기울기 소실) 진단을 위해 Layer-wise Gradient RMS(층별 기울기 제곱평균제곱근), 앞·뒤 은닉층 기울기 비율, ReLU Zero Fraction(렐루 0 비율), Weight Update(가중치 갱신량)를 별도로 기록한다. 양자 보조 방식의 기울기는 학습에 사용하지 않으며 필요한 경우 별도 진단으로만 측정한다. 목표값: 목표 학습 손실은 0.65, 0.55, 0.20이며 미도달도 기록한다.
④ 양자컴퓨팅 적용 방법	Quantum 적용: Qiskit Statevector Simulator(Qiskit 상태벡터 시뮬레이터)의 1 Qubit(큐비트)에 H Gate(하다마드 게이트)를 적용하고 Computational Basis Measurement(계산 기저 측정)의 표본으로 ±1 방향을 만든다. Quantum SPSA(양자 방향 SPSA)는 고전 손실 두 번을 평가해 기울기를 추정한다. Quantum-Direction COBYLA(양자 방향 COBYLA)는 생성한 방향을 고정하고 계수 1개를 최적화한다.

	비교군: Backpropagation SGD(역전파 기반 SGD) Backpropagation Adam(역전파 기반 Adam) Classical SPSA(고전 SPSA) Full-Parameter COBYLA(전체 가중치 COBYLA): 깊은 모델의 실행 여부를 선택한다. Classical-Direction COBYLA(난수 방향 COBYLA) : 난수로 탐색 방향을 생성한다. Quantum SPSA(양자 방향 SPSA) : 1차원 탐색 구조를 사용한다. Quantum-Direction COBYLA(양자 방향 COBYLA) : 1차원 탐색 구조를 사용한다.
⑤ 구현 및 제출물	구현 : Python(파이썬)·PyTorch(파이토치) 기반의 Classical Neural Network(고전 신경망) Python 기반의 ModelBuilder(모델 구성기)·BatchRunner(배치 실행기) SciPy COBYLA(사이파이 COBYLA), Qiskit(큐이스킷) 회로를 사용한 Quantum Optimizer 각 비교군의 실험은 메모리 할당 및 해제 등의 독립성을 보장하기 위해서 별도 Process(프로세스)의 Worker 에서 실행한다. 제출물: Google Colab(구글 코랩)에서 실행 가능한 Notebook(노트북, .ipynb) Worker Code(워커 코드) 시간대별·단계별 Memory Trace(메모리 변화 기록), 층별 기울기·가중치 갱신 진단, 목표 도달 여부 및 오류를 CSV(쉼표 구분 파일) 과 Graph(그래프)
⑥ 검증 및 성공 기준	① 동일 모델·데이터·초기값·Seed(시드)를 확인하고 성공·실패·미실행을 분리한다. ② Seed 42·43·44를 목표로 반복해 개별값, Mean(평균), Standard Deviation(표준편차)을 제시한다. ③ 동일 시간 예산, 동일 Forward Evaluation(순전파 평가) 예산, 동일 목표 품질 도달 비용을 각각 비교한다. Backward Pass(역전파 연산)와 Circuit Sampling(회로 표본 수집) 비용을 함께 보고한다. ④ 작은 모델은 전체 가중치 COBYLA를 포함하고, 깊은 모델은 전체 가중치 COBYLA 실행 상태를 명시한다. CPU RSS(중앙처리장치 상주 메모리)는 1ms Sampling(표본 수집)의 최대값이며 XPU Memory(XPU 메모리)는 할당기 기준이다. ⑤ Layer-wise Gradient RMS(층별 기울기 제곱평균제곱근), 앞·뒤 은닉층 비율과 ReLU Zero Fraction(렐루 0 비율)을 참고하고 미세 값은 float64(64비트 부동소수점) 통계와 가중치 원소 변화로 재확인한다. ⑥ Memory Measurement(메모리 측정): 별도 프로세스에서 학습 전과 학습 중 CPU RSS, XPU Allocated/Reserved Memory(XPU 할당·확보 메모리)를 기록한다. Forward Pass(순전파), Backward Pass(역전파), Parameter Update(파라미터 갱신), Quantum Sampling(양자 표본 수집) 전후와 시간별 값을 측정한다. 시작값·증가량·측정 범위를 구분하며 CPU와 XPU는 별도로 보고한다. ⑦ Gradient Diagnostic(기울기 진단): SGD·Adam 학습 초기와 일정 간격의 층별 기울기 RMS, 앞·뒤 은닉층 기울기 비율, ReLU Zero Fraction(렐루 0 비율), Layer-wise Weight Update RMS(층별 가중치 갱신량)를 기록한다. 진단용 역전파는 양자 보조 방식의 학습 시간·메모리 측정과 분리한다.

	⑧ 반복 및 상태: Seed 42·43·44의 개별값과 평균·표준편차를 제시한다. 실행 완료·실패·미실행과 Optimizer Termination Status(최적화 종료 상태), 최대 평가 횟수 도달 여부 및 오류를 분리해 기록하며 미실행 값을 정상 측정값으로 대체하지 않는다. 성공 기준: 실험을 재현하고 품질·시간·메모리·평가 횟수의 차이와 한계를 설명한다. 양자 방식이 더 좋은 결과를 내는 것은 필수 조건이 아니다.

피드백 받은 내용
교수자 피드백 및 변경 사항을 기록할 예정.